Lecture 27 — NP-Completeness (Part 3)

COMPSCI 308 — Design and Analysis of Algorithms

Xing Shi Cai https://newptcai.gitlab.io

Duke Kunshan University, Kunshan, China

Jan-Mar 2026

Summary

ITA 34.3 NP-Completeness and Reducibility

NP-Completeness

Circuit satisfiability

ITA 34.4 NP-Completeness Proofs

A Recipe for Proving NP-Completeness

Formula Satisfiability

3-CNF Satisfiability

Assignments[1]

Practice makes perfect!

📖 Required Readings:

- Section 34.3 except pp. 1073-1077.
- Section 34.4: 3-CNF Satisfiability.

✏️ Required Exercises:

- Exercises 34.3: 1-3.
- Exercises 34.4: 1-2, 5.

[1] 💣 Assignments will not be collected; however, quiz problems will be selected from them. (This includes both Readings and Exercises.)

ITA 34.3 NP-Completeness and Reducibility

ITA 34.3 NP-Completeness and Reducibility

NP-Completeness

Understanding NP-Completeness

A language $L \subseteq \{0,1\}^*$ is NP-complete if:

1. L is in NP, and
2. For every L' in NP, $L' \leq_p L$.

Reduce in polynomial time

Understanding NP-Completeness

A language $L \subseteq \{0,1\}^*$ is NP-complete if:

1. L is in NP, and
2. For every L' in NP, $L' \leq_p L$.

If a language L satisfies the second condition but not necessarily the first, we call L as NP-hard.

The class of NP-complete languages is denoted by NPC.

$\neq$ Non-Play-Character

$\neq$ National People's Congress.

Theorem 34.4

😲 If there exists a problem in NPC which is polynomial-time solvable, then *all* problems in NP are polynomial-time solvable, i.e.,

$$P = NP$$

Theorem 34.4

😮 If there exists a problem in NPC which is polynomial-time solvable, then *all* problems in NP are polynomial-time solvable, i.e.,

$$P = NP$$

only two possible cases

If there exists a problem in NP which is *not* polynomial-time solvable, i.e.,

$$P \neq NP$$

then *no* problem in NPC is polynomial-time solvable.

Theorem 34.4

😲 If there exists a problem in NPC which is polynomial-time solvable, then *all* problems in NP are polynomial-time solvable, i.e.,

$$\mathrm{P} = \mathrm{NP}$$

If there exists a problem in NP which is *not* polynomial-time solvable, i.e.,

$$\mathrm{P} \neq \mathrm{NP}$$

then *no* problem in NPC is polynomial-time solvable.

😩 This seemingly easy problem has been open for 50 years!

🤔 How do we know that NPC is not empty?

ITA 34.3 NP-Completeness and Reducibility

Circuit satisfiability

Example — Adding Two Bits

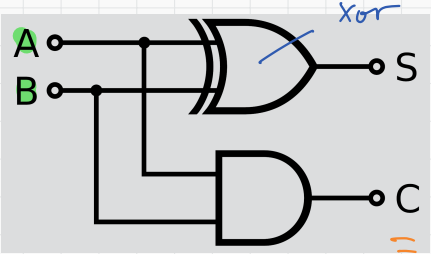
Computers use electronic circuits like the one shown below to perform binary addition.

← 0/1

The inputs A and B represent the bits to be added.

The outputs C and S represent the result of the addition.

Inputs Outputs



$= \text{Xor}(A,B)$

$= \begin{cases} 0 & (A = B) \\ 1 & (A \neq B). \end{cases}$

$= A \wedge B.$

A	B	C	S	
0	0	0	0	= 0.
0	1	0	1	= 1
1	0	0	1	= 1
1	1	1	0	= 2

Table 1: Truth table of the circuit.

$A + B.$

Figure 1: Circuit for adding two bits.

Boolean Combinational Circuits

Boolean combinational circuits consist of interconnected boolean combinational elements.

0/1 0/1

These are circuits with fixed numbers of boolean inputs and outputs, executing well-defined functions.

A Boolean input is either 0 (FALSE) or 1 (TRUE).

Boolean Combinational Circuits

Boolean combinational circuits consist of interconnected boolean combinational elements.

These are circuits with fixed numbers of boolean inputs and outputs, executing well-defined functions.

A Boolean input is either 0 (FALSE) or 1 (TRUE).

These circuits consists of using logic gates:

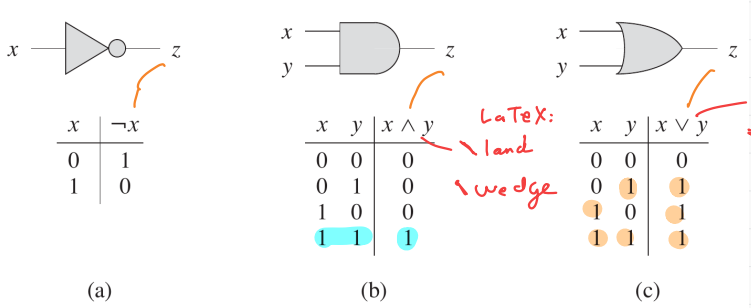
- NOT gate: Flips the binary input x. $0 \to 1$, $1 \to 0$.
- AND gate: Outputs 1 if both binary inputs x and y are 1. $x \wedge y$.
- OR gate: Outputs 1 if either binary input x or y is 1. $x \vee y$

Boolean Combinational Circuits

Boolean combinational circuits consist of interconnected boolean combinational elements.

These are circuits with fixed numbers of boolean inputs and outputs, executing well-defined functions.

A Boolean input is either 0 (FALSE) or 1 (TRUE).



x	$\neg x$
0	1
1	0

(a)

x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

(b)

x	y	$x \vee y$
0	0	0
0	1	1
1	0	1
1	1	1

(c)

Figure 2: Logic Gates

Circuit Satisfiability

The Circuit Satisfiability Problem is a decision problem which asks if there exist inputs which make a boolean combinatorial circuit C output 1.

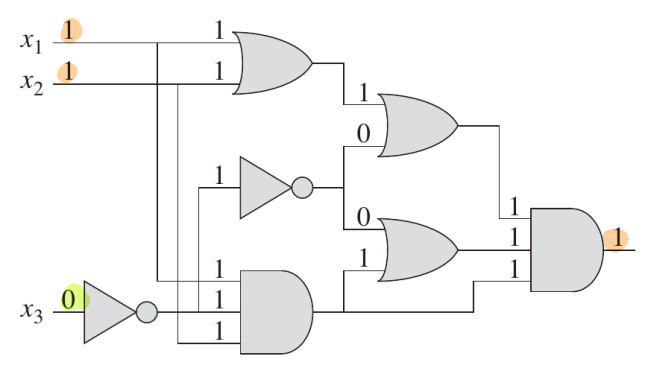


Figure 3: 📌 Example — A satisfiable circuit.

Circuit Satisfiability

The Circuit Satisfiability Problem is a decision problem which asks if there exist inputs which make a boolean combinatorial circuit C output 1.

We define the formal language:

binary string encoding of a circuit c.

$$\text{CIRCUIT-SAT} = \{\langle C \rangle : C \text{ is a satisfiable boolean combinatorial circuit}\}$$

😲 This is the first known problem in NPC!

🌱 Proving any other problem is in NPC needs this!

Theorem 34.7

It follows from the following two lemma that CIRCUIT-SAT is NP-complete:

Lemma 34.5

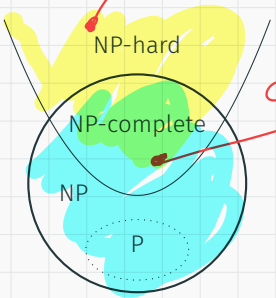
The circuit-satisfiability problem is in NP.

Lemma 34.6

The circuit-satisfiability problem is NP-Hard.

All NP problem can be reduced to this problem

This is not in NP.



CIRCUIT-SAT

Figure 3: The case P ≠ NP

😜 The proofs are not required.

👀 We will focus on showing other problems are in NPC.

ITA 34.4 NP-Completeness Proofs

ITA 34.4 NP-Completeness Proofs

A Recipe for Proving NP-Completeness

Lemma 34.8

If L is in NP and there exists $L' \in$ NPC such that $L' \leq_p L$, then L is in NP-complete.

Kind of saying L is more general than L'.

Argument:

For any NP problem L'',

we can reduce it first to L' (it's in NPC).

the reduce it to L.

$L'' \leq_p L' \leq_p L$

Any NP In NPC Also in NPC.

Lemma 34.8

If L is in NP and there exists $L' \in$ NPC such that $L' \leq_p L$, then L is in NP-complete.

💡 This allows us to prove many many many problems are NP-complete with the following recipe:

1. Prove that $L \in$ NP.
2. Select problem $L' \in$ NPC.
3. Pick $f : L' \mapsto L$.
4. Show that $x \in L'$ if and only if $f(x) \in L$.
5. Prove that f is polynomial-time computable.

🤪 Karp's 21 NP-Complete Problems

In 1972, Richard Karp used this method to show that 21 problems combinatorial and graph theory are NP-complete.

These are known as Karp's 21 NP-complete problems.

🏆 Karp won 1985 Turing Award.

Figure 4: Richard Karp (1935-). 🇺🇸 computer scientist.

ITA 34.4 NP-Completeness Proofs

Formula Satisfiability

📌 Example of Boolean Formula

Consider the statement —

False only when $x_1 = x_2 = 0$.

x_1 ✓ x_2

Either I get less than 90 points, or I get an A.

🍰 In which case is this a false statement?

I get more than 90 and I get less than A.

📌 Example of Boolean Formula

Consider the statement —

Either I get less than 90 points, or I get an A.

To see thins clearly, we can define two boolean variables

- x_1 — I get less than 90 points
- x_2 — I get an A

Then the statement becomes

$$x_1 \vee x_2$$

🍰 In which case is this a false statement?

$x_1 = x_2 = 0.$

📌 Example of Boolean Formula

Consider the statement —

Either I get less than 90 points, or I get an A.

🍰 In which case is this a false statement?

We can find the answer by looking at the following truth table

x_1	x_2	$x_1 \vee x_2$
0	0	0
0	1	1
1	0	1
1	1	1

Boolean Formula

A boolean formula ϕ, is described by:

- Variables: A set of boolean variables, $\{x_1, x_2, \dots, x_n\}$.
- Connectives: Boolean functions, such as $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT), $\rightarrow$ (implies), $\Leftrightarrow$ (iff).
- Parentheses: Used to group expressions.

📌 Examples include:

- $x_1 \vee x_2$ $\quad$ x_1 or x_2
- $x_1 \wedge x_2$ $\quad$ x_1 and x_2
- $(x_1 \rightarrow x_2) \vee (\neg(\neg x_1 \Leftrightarrow x_3))$

$= 1$

Satisfying Assignment (not necessarily unique).

$x_1 = 1, \quad x_2 = 0$

$x_1 = \underline{1}, \quad x_2 = \underline{1}.$

$x_1 = \underline{1}, \quad x_2 = \underline{1}, \quad x_3 = \underline{1}$

$(x_1 \rightarrow x_2) = 1$

$x_1 = 0, \quad x_2 = ?, \quad x_3 = ?$

Boolean Formulas and SAT Problem

A truth assignment is a set of values (1/0) assigned to the variables of ϕ.

A satisfying assignment is a truth assignment that makes the formula evaluate to 1.

Satisfiability (SAT) is the decision problem to determine if a boolean formula has a *satisfying assignment*.

In formal language terms,

$$\text{SAT} = \{\langle\phi\rangle : \phi \text{ is a satisfiable boolean formula}\}.$$

Boolean Formulas and SAT Problem

A truth assignment is a set of values (1/0) assigned to the variables of ϕ.

A satisfying assignment is a truth assignment that makes the formula evaluate to 1.

Satisfiability (SAT) is the decision problem to determine if a boolean formula has a *satisfying assignment*.

Example : of formula not in SAT

$x_1 \wedge (\neg x_1)$ ← Always 0.

In formal language terms,

$$\text{SAT} = \{\langle\phi\rangle : \phi \text{ is a satisfiable boolean formula}\}.$$

Example

For instance, consider the formula:

0/1 $x_2 = 0$

$$\phi = ((x_1 \to x_2) \vee \neg((\neg x_1 \Leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$$

1 0 0 1 1.

Is $\phi \in$ SAT?

Theorem 34.9

Satisfiability of boolean formulas is NP-complete.

✏️ Proof Sketch — Step 1 — Prove that SAT $\in$ NP.

📌 Example

Consider the formula:

$$\phi = ((x_1 \to x_2) \lor \neg((\neg x_1 \Leftrightarrow x_3) \lor x_4)) \land \neg x_2$$

🍰 Is this a satisfying assignment?

$$x_1 = 0, \quad x_2 = 0, \quad x_3 = 1, \quad x_4 = 1$$

Since a formula has polynomially many connectives, it takes polynomial time to check if an assignment is satisfying.

Theorem 34.9

Satisfiability of boolean formulas is NP-complete.

✏️ Proof Sketch — Step 2 — Prove that SAT is NP-hard by showing that CIRCUIT-SAT $\leq_p$ SAT.

It can be done by giving each wire a variable and express each gate with an $\Leftrightarrow$ formula, which we call a clause.

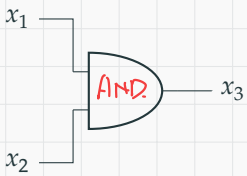


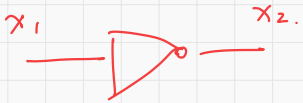
Figure 5: 📌 This and gate is equivalent to a clause $(x_1 \wedge x_2) \Leftrightarrow x_3$.

Theorem 34.9

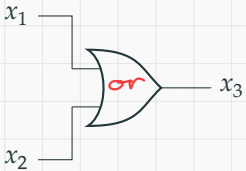
Satisfiability of boolean formulas is NP-complete.

✏️ Proof Sketch — Step 2 — Prove that SAT is NP-hard by showing that CIRCUIT-SAT $\leq_p$ SAT.

It can be done by giving each wire a variable and express each gate with an $\Leftrightarrow$ formula, which we call a clause.



$(\neg x_1) \Leftrightarrow x_2$.



$(x_1 \vee x_2) \Leftrightarrow x_3$.

Figure 5: 🍰 Which clause is equivalent to this or gate?

Theorem 34.9

Satisfiability of boolean formulas is NP-complete.

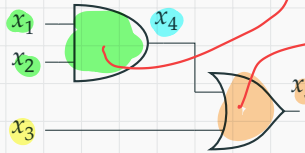
✏️ Proof Sketch — Step 2 — Prove that SAT is NP-hard by showing that CIRCUIT-SAT $\leq_p$ SAT.

It can be done by giving each wire a variable and express each gate with an $\Leftrightarrow$ formula, which we call a clause.

A formula can be produced by taking conjunction (AND) of the output variable and the clauses.

For the following circuit, this gives

$$\phi = x_5 \wedge (x_5 \Leftrightarrow x_3 \vee x_4) \wedge (x_4 \Leftrightarrow x_1 \wedge x_2)$$



Use clause

$x_5 = 1$

Theorem 34.9

Satisfiability of boolean formulas is NP-complete.

✏️ Proof Sketch —

A formula can be produced by taking conjunction (AND) of the output variable and the clauses.

For the following circuit, this gives

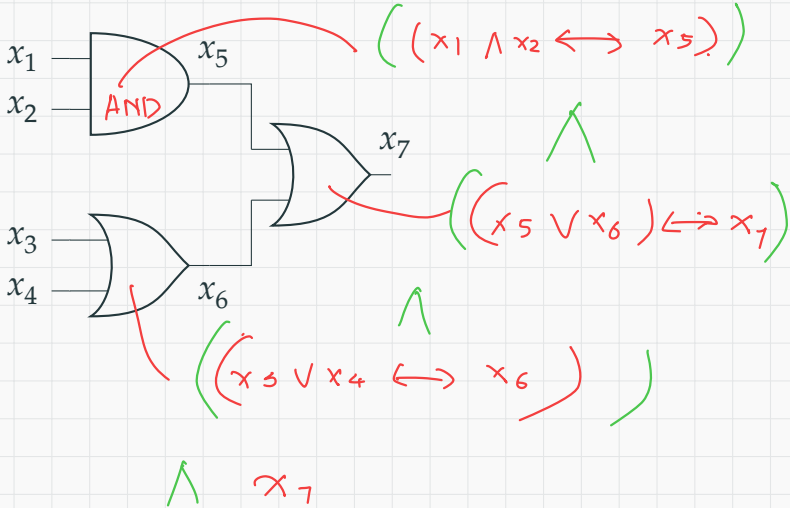
$$\phi = x_5 \wedge (x_5 \Leftrightarrow x_3 \vee x_4) \wedge (x_4 \Leftrightarrow x_1 \wedge x_2)$$

Constructing ϕ takes polynomial time.

The circuit is satisfiable if and only if ϕ is satisfiable.

Thus we have CIRCUIT-SAT $\leq_p$ SAT.

Can you convert this circuit to an equivalent boolean formula?



🤪 Applications of SAT

Many real-world problems can be efficiently formulated as SAT (Boolean Satisfiability) problems:

1. 👆 Sudoku puzzles solving
2. Formal verification in hardware and software design
3. Optimising scheduling tasks
4. Addressing certain 👆 combinatorial challenges in mathematics

Modern SAT solvers (software) are remarkably efficient.

ITA 34.4 NP-Completeness Proofs

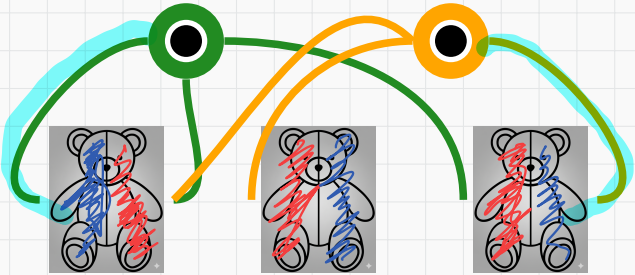
3-CNF Satisfiability

Aliens and Teddy Bears

Three 🧸 can be coloured red on one side and blue on the other. (You can choose which side is red and which is blue.)

Each of two of 3-armed 👽 grabs 3 teddy bear hands.

🍰 Can we colour the 🧸 such that every 👽 is holding at least one blue hand?



Aliens and Teddy Bears

This is equal event to ask if

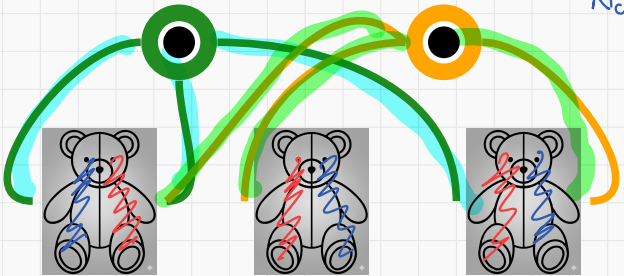
or and or

$$\underbrace{(x_1 \vee \neg x_1 \vee x_3)}_{3} \wedge \underbrace{(\neg x_1 \vee x_2 \vee \neg x_3)}_{3}$$

is satisfiable, where x_i is true if bear i is blue on the left.

A boolean formula like this is called a 3-CNF formula.

Conjunctive Normal Form.



$x_1 = 1$ $x_2 = 0$ $x_3 = 0$

3-CNF

A literal is either a variable or its negation.

A formula in conjunctive normal form (CNF) is an AND of clauses, each being the OR of one or more literals.

🍰 Which of the following is a CNF formula?

$$x_1 \wedge (x_3 \vee x_2 \vee x_4), \qquad x_1 \wedge (x_1 \vee \neg x_2),$$

3-CNF

A literal is either a variable or its negation.

A formula in conjunctive normal form (CNF) is an AND of clauses, each being the OR of one or more literals.

A formula is in 3-conjunctive normal form, or 3-CNF, if each clause has *exactly three distinct literals*.

🍰 Which of the following is a 3-CNF formula?

$$(x_1 \lor \neg x_1 \lor \neg x_2) \land (x_3 \lor x_2 \lor x_4)$$

and

$$(x_4 \land \neg x_1 \land \neg x_2) \lor (x_3 \land x_2 \land x_4)$$

is in 3-CNF.

3-CNF

A literal is either a variable or its negation.

A formula in conjunctive normal form (CNF) is an AND of clauses, each being the OR of one or more literals.

A formula is in 3-conjunctive normal form, or 3-CNF, if each clause has *exactly three distinct literals*.

The 3-CNF-SAT is a decision problem which asks if a formula in 3-CNF is satisfiable.

Theorem 34.10

3-CNF-SAT is NP-complete.

✏️ Proof Sketch — Note that 3-CNF-SAT is a specific case of SAT.

📌 **Example**

Verify

$$x_1 = 0, \qquad x_2 = 1, \qquad x_3 = 0$$

a satisfying assignment for

$$x_1 \vee \neg x_2 \vee x_3$$

Theorem 34.10

3-CNF-SAT is NP-complete.

✏️ Proof Sketch — Note that 3-CNF-SAT is a specific case of SAT.

📌 **Example**

Verify

$$x_1 = 0, \qquad x_2 = 1, \qquad x_3 = 0$$

a satisfying assignment for

$$x_1 \vee \neg x_2 \vee x_3$$

Since we can verify an assignment for SAT in polynomial time, we can also verify an assignment for 3-CNF-SAT in polynomial time.

In other words, 3-CNF-SAT $\in$ NP.

Theorem 34.10

3-CNF-SAT is NP-complete.

✏️ Proof Sketch — Note that 3-CNF-SAT is a specific case of SAT.

Since we can verify an assignment for SAT in polynomial time, we can also verify an assignment for 3-CNF-SAT in polynomial time.

In other words, 3-CNF-SAT $\in$ NP.

So we only need to show that

$$\text{SAT} \leq_p \text{3-CNF-SAT}$$

In other words, we need to show that any boolean formula can be converted to an equivalent formula in 3-CNF.

Conversion to 3-CNF

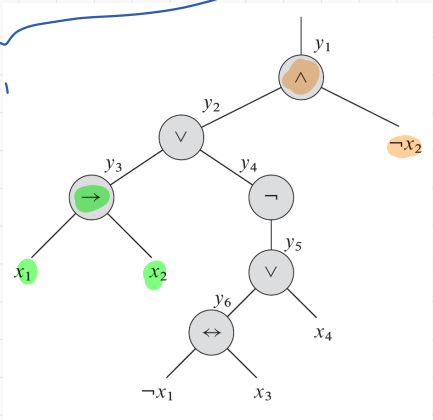
We will convert the following formula into an equivalent 3-CNF formula.

$$\phi = ((x_1 \to x_2) \lor \neg((\neg x_1 \leftrightarrow x_3) \lor x_4)) \land \neg x_2$$

Conversion to 3-CNF

We start by convert ϕ into a tree as shown below —

$$\phi = \underbrace{\underbrace{((x_1 \to x_2)}_{y_3} \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4))}_{y_1} \wedge \neg x_2$$

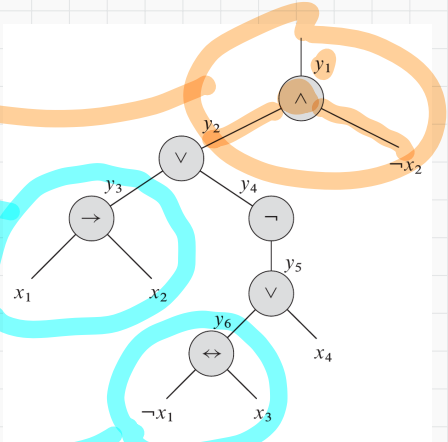


Conversion to 3-CNF

The next step is to convert the tree into a CNF:

$$
\begin{aligned}
\phi' = y_1 & \\
& \wedge (y_1 \leftrightarrow (y_2 \wedge \neg x_2)) \\
& \wedge (y_2 \leftrightarrow (y_3 \vee y_4)) \\
& \wedge (y_3 \leftrightarrow (x_1 \rightarrow x_2)) \\
& \wedge (y_4 \leftrightarrow \neg y_5) \\
& \wedge (y_5 \leftrightarrow (y_6 \vee x_4)) \\
& \wedge (y_6 \leftrightarrow (\neg x_1 \leftrightarrow x_3))
\end{aligned}
$$

We will convert each clause to a 3-CNF formula.



Conversion to 3-CNF

The final step is to convert clauses to a 3-CNF formula.

For example the clause $c_1 = y_1 \leftrightarrow (y_2 \wedge \neg x_2)$ has truth table

y_1	y_2	x_2	c_1	$\neg c_1$
1	1	1	0	1
1	1	0	1	0
1	0	1	0	1
1	0	0	0	1
0	1	1	1	0
0	1	0	0	1
0	0	1	1	0
0	0	0	1	0

This implies that

$$\neg c_1 = (y_1 \wedge y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge \neg x_2) \vee (\neg y_1 \wedge y_2 \wedge \neg x_2).$$

Conversion to 3-CNF

This implies that

$$\neg c_1 = (y_1 \wedge y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge \neg x_2) \vee (\neg y_1 \wedge y_2 \wedge \neg x_2).$$

We can then apply De-Morgan's laws

$$\neg(a \vee b) = \neg a \wedge \neg b, \qquad \neg(a \wedge b) = \neg a \vee \neg b$$

Conversion to 3-CNF

This implies that

$$\neg c_1 = (y_1 \wedge y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge \neg x_2) \vee (\neg y_1 \wedge y_2 \wedge \neg x_2).$$

We can then apply De-Morgan's laws

$$\neg(a \vee b) = \neg a \wedge \neg b, \qquad \neg(a \wedge b) = \neg a \vee \neg b$$

to get

$$\begin{aligned} & c_1 \\ &= \neg(y_1 \wedge y_2 \wedge x_2) \wedge \neg(y_1 \wedge \neg y_2 \wedge x_2) \wedge \neg(y_1 \wedge \neg y_2 \wedge \neg x_2) \wedge \neg(\neg y_1 \wedge y_2 \wedge \neg x_2) \\ &= \underbrace{(\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee x_2) \wedge (y_1 \vee \neg y_2 \vee x_2)}_{\text{3-CNF.}} \end{aligned}$$

This is a 3-CNF formula!

Wooclap

Convert $c = x_1 \to (x_2 \to x_3)$ to a 3-CNF formula given the following truth table

x_1	x_2	x_3	c
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

Clause With Only Two Literals

The boolean formula

$$\phi = x_1 \vee x_2$$

can be converted to 3-CNF by adding a new literal y_1 —

$$\phi' = (x_1 \vee x_2 \vee y_1) \wedge (x_1 \vee x_2 \vee \neg y_1)$$

Clause With Only Two Literals

The boolean formula

$$\phi = x_1 \vee x_2$$

can be converted to 3-CNF by adding a new literal y_1 —

$$\phi' = (x_1 \vee x_2 \vee y_1) \wedge (x_1 \vee x_2 \vee \neg y_1)$$

It is easy to see that the ϕ and ϕ' are equivalent —

- If $y_1 = 0$, then

$$\phi' = (x_1 \vee x_2 \vee 0) \wedge (x_1 \vee x_2 \vee 1) = x_1 \vee x_2 = \phi$$

Clause With Only Two Literals

The boolean formula

$$\phi = x_1 \vee x_2$$

can be converted to 3-CNF by adding a new literal y_1 —

$$\phi' = (x_1 \vee x_2 \vee y_1) \wedge (x_1 \vee x_2 \vee \neg y_1)$$

It is easy to see that the ϕ and ϕ' are equivalent —

- If $y_1 = 0$, then

$$\phi' = (x_1 \vee x_2 \vee 0) \wedge (x_1 \vee x_2 \vee 1) = x_1 \vee x_2 = \phi$$

- If $y_1 = 1$, then again

$$\phi' = (x_1 \vee x_2 \vee 1) \wedge (x_1 \vee x_2 \vee 0) = x_1 \vee x_2 = \phi$$

Clause With Only One Literal

The boolean formula

$$\phi = x_1$$

can be converted to 3-CNF by adding tow literals y_1 and y_2 —

$$\phi' = (x_1 \vee y_1 \vee y_2) \wedge (x_1 \vee y_1 \vee \neg y_2) \wedge (x_1 \vee \neg y_1 \vee y_2) \wedge (x_1 \vee \neg y_1 \vee \neg y_2)$$

🍰 Is there an easy way to check that ϕ and ϕ' are indeed equivalent?

🤪 NP-Complete Proofs

Using the same proof method, we can show all the problems below are NP-complete.

But this is the end of the course.

You will have to learn how the proofs work in graduate school or buy yourself.

☘ Good luck.

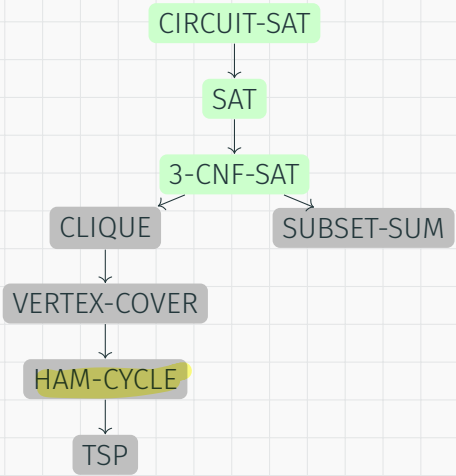


Figure 5: The structure of NP-completeness proofs

What are your main takeaways today? Any questions?

Please be honest... but maybe a little gentle? (sweating)
COURSE EVALUATION TODAY!
Hmm... "Instructor nervousness": 10/10.